МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №8
по курсу «Дискретный анализ»
на тему «Жадные алгоритмы»

Выполнил: Н. В. Храмов
Группа: М8О-208БВ-24
Преподаватель: Макаров Н.К.
Проверил: Сахарин Н.А.
Дата: 26.10.2026
Оценка:
Подпись:

Москва, 2026

Постановка задачи

Вариант 4. «Откорм бычков». Имеются M пищевых добавок, каждая из которых содержит N действующих веществ. Для добавки известны количества веществ a_1, …, a_n и цена одного мешка. Воздействие добавки равно $c_1a_1 + \ldots + c_na_n$, где коэффициенты c_i неизвестны и одинаковы для всех добавок.

Измерение воздействия одной добавки требует одного мешка. Необходимо выбрать набор добавок минимальной суммарной стоимости, измерения которого позволят однозначно определить все N коэффициентов. Если подходящего набора нет, следует сообщить об этом.

Формат ввода

В первой строке заданы M и N. Следующие M строк содержат N количеств веществ и цену мешка соответствующей добавки. Количества и цены — неотрицательные целые числа не больше 50. Номера добавок соответствуют порядку строк и начинаются с единицы.

В тексте условия указано $M \le N$, однако во втором примере $M = 3$, $N = 2$. Реализация допускает оба соотношения. При $M < N$ определить все коэффициенты невозможно. Ограничение времени — 10 секунд, памяти — 1 ГБ.

Формат вывода

Если определить коэффициенты невозможно, выводится −1. Иначе выводятся номера выбранных добавок в порядке возрастания их номеров во входных данных. При нескольких наборах минимальной стоимости подходит любой из них.

Алгоритм решения задачи

Каждое измерение задаёт линейное уравнение относительно коэффициентов c_i. Они определяются однозначно тогда и только тогда, когда матрица выбранных составов имеет ранг N. Поэтому требуется найти базис строк минимального суммарного веса, где вес строки равен цене мешка.

Используется жадный алгоритм: добавки сортируются по возрастанию цены, затем очередная добавка включается в ответ только в том случае, если её вектор линейно независим от уже выбранных. Проверка выполняется последовательным исключением Гаусса. После выбора N независимых строк поиск завершается.

Этапы решения

1. Чтение данных. Для каждой добавки сохраняются вектор количеств веществ, цена и исходный номер. Если добавок меньше, чем неизвестных коэффициентов, возвращается пустой результат, который main выводит как −1.

2. Сортировка. Метод Select упорядочивает добавки по цене. При равной цене порядок произвольный, поскольку задача допускает несколько оптимальных ответов.

3. Проверка независимости. Метод Add вычитает из очередного вектора кратные строки текущего базиса, обнуляя их опорные координаты. Если остаток нулевой, добавка зависима и пропускается. Иначе опорной выбирается координата с наибольшим модулем, строка нормируется и сохраняется в базисе.

4. Формирование ответа. Номер каждой принятой добавки записывается в результат. При достижении размера N номера сортируются по возрастанию. Если просмотрены все добавки, а N независимых строк не найдено, выводится −1.

Доказательство корректности

В точной арифметике вычитание линейных комбинаций строк базиса не меняет принадлежность вектора их линейной оболочке. После исключения всех опорных координат нулевой остаток означает зависимость. Ненулевой остаток содержит новую опорную координату и увеличивает ранг на единицу. Следовательно, алгоритм сохраняет независимость выбранных строк.

Докажем оптимальность жадного выбора обменным аргументом. Пусть уже выбранное множество S содержится в некотором оптимальном базисе B. На очередном шаге выбирается самая дешёвая из оставшихся строк v, независимых от S. Если v принадлежит B, свойство сохраняется.

Иначе представим v через строки B. Поскольку v не лежит в линейной оболочке S, в этом представлении есть строка u из B вне S с ненулевым коэффициентом. Замена u на v сохраняет базис. Строка u независима от S и не могла быть ранее пропущена как зависимая: зависимость сохраняется при расширении выбранного множества. Поэтому цена v не больше цены u. После замены получаем оптимальный базис, содержащий S и v.

Изначально пустое множество содержится в любом оптимальном базисе. По индукции после N успешных шагов выбранный базис имеет минимальную стоимость. Если ранг всех строк меньше N, однозначно определить коэффициенты невозможно. Поскольку цены неотрицательны, любой достаточный набор можно сократить до базиса без увеличения стоимости, поэтому поиск среди базисов решает исходную задачу.

Анализ сложности

Сортировка требует O(M log M). При проверке одного вектора выполняется не более N исключений, каждое обрабатывает N координат, поэтому проверка стоит $O(N^2)$. Общая верхняя оценка времени — $O(M \log M + MN^2)$. После нахождения базиса просмотр прекращается. Сортировка N номеров ответа укладывается в эту оценку при $M \geq N$.

Исходные составы занимают O(MN) памяти, базис — $O(N^2)$, ответ — O(N). Общая оценка — $O(MN + N^2)$. На ветви решения при $M \geq N$ её можно записать как O(MN). При $M < N$ программа возвращает невозможность сразу после чтения данных.

Векторы передаются в метод Add через std::move, поэтому отдельная копия состава для каждой проверки не создаётся. Для результата заранее резервируется N элементов. Цена хранится в int: суммарная стоимость в реализации не вычисляется, поскольку для жадного выбора достаточно сравнения отдельных цен.

Вычисления Гаусса используют long double и порог 10^{-12} для нулевого остатка. Выбор максимальной по модулю опорной координаты уменьшает влияние деления на малые значения. Доказательство относится к точной арифметике; для плохо обусловленных матриц вычисления с плавающей точкой могут давать погрешность. По сообщению пользователя, реализация прошла все тесты проверяющей системы.

Структура программы

Supplement — структура данных одной добавки: substances хранит состав, price — цену, index — исходный номер. Она не содержит алгоритмических действий.

LinearBasis инкапсулирует математическую часть: размерность, строки базиса, опорные координаты и порог сравнения. Публичный метод Add проверяет независимость и при успехе расширяет базис. Класс ничего не знает о ценах, номерах добавок и потоках ввода и вывода.

SupplementSelector отвечает за бизнес-логику выбора: сортирует добавки по цене, обращается к LinearBasis и возвращает номера минимального достаточного набора. Метод Select объявлен const; данные для отдельного запуска поступают по значению, а из main передаются перемещением.

Функция main читает данные, вызывает Select и форматирует результат. Алгоритм не зависит от потоков и может вызываться отдельно. Такое разделение соответствует инкапсуляции и принципу единственной ответственности. Наследование и отдельные классы для простого ввода и вывода здесь не требуются.

Исходный код

```
#include <algorithm>
#include <cmath>
#include <iostream>
#include <utility>
#include <vector>

struct Supplement {
    std::vector<long double> substances;
    int price;
    int index;
};

class LinearBasis {
public:
    explicit LinearBasis(int dimension) : dimension_(dimension) {}

    bool Add(std::vector<long double> row) {
        for (std::size_t i = 0; i < rows_.size(); ++i) {
            const int pivot = pivots_[i];
            const long double factor = row[pivot];
            for (int j = 0; j < dimension_; ++j) {
                row[j] -= factor * rows_[i][j];
            }
            row[pivot] = 0;
        }

        int pivot = -1;
        for (int j = 0; j < dimension_; ++j) {
            if (pivot == -1 || std::abs(row[j]) > std::abs(row[pivot])) {
                pivot = j;
            }
        }
        if (pivot == -1 || std::abs(row[pivot]) < kEpsilon) {
            return false;
        }

        const long double divisor = row[pivot];
        for (long double& value : row) {
            value /= divisor;
        }
        row[pivot] = 1;
        pivots_.push_back(pivot);
        rows_.push_back(std::move(row));
        return true;
    }

private:
    static constexpr long double kEpsilon = 1e-12L;
    int dimension_;
    std::vector<int> pivots_;
    std::vector<std::vector<long double>> rows_;
};

class SupplementSelector {
public:
    explicit SupplementSelector(int substance_count)
        : substance_count_(substance_count) {}
```

```
    std::vector<int> Select(std::vector<Supplement> supplements) const {
        if (supplements.size() < static_cast<std::size_t>(substance_count_)) {
            return {};
        }
        std::sort(supplements.begin(), supplements.end(),
                  [](const Supplement& lhs, const Supplement& rhs) {
                      return lhs.price < rhs.price;
                  });

        LinearBasis basis(substance_count_);
        std::vector<int> selected;
        selected.reserve(substance_count_);
        for (Supplement& supplement : supplements) {
            if (basis.Add(std::move(supplement.substances))) {
                selected.push_back(supplement.index);
                if (selected.size() == static_cast<std::size_t>(substance_count_)) {
                    std::sort(selected.begin(), selected.end());
                    return selected;
                }
            }
        }
        return {};
    }

private:
    int substance_count_;
};

int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);

    int supplement_count;
    int substance_count;
    if (!(std::cin >> supplement_count >> substance_count)) {
        return 0;
    }

    std::vector<Supplement> supplements(supplement_count);
    for (int i = 0; i < supplement_count; ++i) {
        Supplement& supplement = supplements[i];
        supplement.substances.resize(substance_count);
        for (long double& amount : supplement.substances) {
            std::cin >> amount;
        }
        std::cin >> supplement.price;
        supplement.index = i + 1;
    }

    SupplementSelector selector(substance_count);
    const std::vector<int> selected = selector.Select(std::move(supplements));
    if (selected.empty()) {
        std::cout << "-1\n";
    } else {
        for (std::size_t i = 0; i < selected.size(); ++i) {
            if (i != 0) {
                std::cout << ' ';
            }
```

```
            std::cout << selected[i];
        }
        std::cout << '\n';
    }
    return 0;
}
```

Тесты

Приведённые результаты.

Тест 1. Пример 1: недостаточный ранг

```
Ввод:
3 3
1 0 2 3
1 0 2 4
2 0 1 2
Вывод:
-1
```

Тест 2. Пример 2: дешёвая альтернатива

```
Ввод:
3 2
2 1 2
1 2 9
1 2 3
Вывод:
1 3
```

Тест 3. Добавок меньше, чем веществ

```
Ввод:
1 2
1 1 5
Вывод:
-1
```

Тест 4. Нулевая строка и бесплатный базис

```
Ввод:
3 2
0 0 0
1 0 0
0 1 0
Вывод:
2 3
```

Тест 5. Пропуск дешёвой зависимой строки

```
Ввод:
4 2
1 0 1
2 0 2
0 1 3
1 1 10
Вывод:
1 3
```

Тест 6. Равные цены

```
Ввод:
3 2
1 0 5
0 1 5
1 1 5
Вывод:
1 2
```

Тест 7. Одно вещество

```
Ввод:
3 1
0 0
2 7
1 3
Вывод:
3
```

Тест 8. Все составы нулевые

```
Ввод:
2 2
0 0 1
0 0 2
Вывод:
-1
```

Проверка решения

В первом примере второй столбец состоит из нулей, а первые две строки совпадают. Ранг меньше трёх, поэтому программа выводит −1. Во втором примере выбираются добавки 1 и 3 стоимостью 2 + 3 = 5; их определитель равен 3, поэтому оба коэффициента определяются однозначно.

В тесте с зависимыми строками добавка 2 дешевле добавки 3, но пропускается: её состав вдвое больше состава добавки 1 и не даёт нового независимого уравнения. Выбираются добавки 1 и 3. Нулевая строка не включается в базис даже при нулевой цене.

Генератор отчёта проверил восемь приведённых тестов и 500 случайных случаев с $1 \le N \le 5$ и $1 \le M \le 8$. Эталон перебирает все подмножества размера N, вычисляет ранг в точной рациональной арифметике Fraction и выбирает минимальную стоимость. Все результаты совпали с эталоном по возможности решения и минимальной стоимости.

Для каждого найденного ответа дополнительно проверялись количество и уникальность номеров, их допустимый диапазон, порядок возрастания и полный ранг выбранных составов. При равных ценах сравнивалась стоимость, а не конкретный набор номеров.

Проверочная сборка выполнена с параметрами g++ -std=c++20 -O2 -Wall -Wextra -Wpedantic -Wconversion. Для режима отправки (make) C++20 в проекте имеется Makefile, создающий исполняемый файл main. По сообщению пользователя, все тесты проверяющей системы пройдены. Локальный полный перебор подтверждает корректность на малых входах, но не является измерением производительности на максимальных размерах, которые в предоставленном условии не указаны.

Вывод

В ходе работы реализовано решение задачи «Откорм бычков» жадным алгоритмом построения базиса минимальной стоимости. Связь измерений с системой линейных уравнений позволяет свести достаточность набора к полному рангу, а последовательное исключение Гаусса — проверять независимость очередной добавки.

Оптимальность жадного выбора обоснована обменом строк базиса. Реализация имеет временную сложность $O(M \log M + MN^2)$ и использует $O(MN + N^2)$ памяти. Математические операции, выбор набора и ввод-вывод разделены. Корректность подтверждена локальными проверками и прохождением тестов проверяющей системы; при оценке численной устойчивости учитывается использование long double.